

Q1) Define a structure in C and C++.

This structure should hold the variables for student type. Compute the size of the structure manually.

ii) use size of function & display the size

cross check if both are same (or) not.

Justify the Answer:-

In C Language :-

```
#include <stdio.h>
```

```
struct student {
```

```
    int id;
```

```
// 4 bytes
```

```
    float marks;
```

```
// 4 bytes
```

```
    char c1;
```

```
// 1 bytes
```

```
    char c2[10];
```

```
// 10 bytes
```

```
};
```

```
void main() {
```

```
    struct student s;
```

```
    printf("size of (s) result: %lu
```

```
bytes\n", sizeof(s));
```

```
}
```


Manual Calculation :-

$$4 + 4 + 1 + 10 = 19 \text{ bytes}$$

Output :-

sizeof() result: 120 bytes

In C++ :-

```
#include <iostream>
```

```
using namespace std;
```

```
struct Data {
```

```
    int i;           // 4 bytes
```

```
    float f;         // 4 bytes
```

```
    char c1;         // 1 bytes
```

```
    char c2[10];     // 10 bytes
```

```
};
```

```
void main() {
```

```
    Data d;
```

```
    cout << "sizeof() function result
```

```
    << sizeof(d) << "bytes" << endl;
```

Manual Calculation :- $4 + 4 + 1 + 10 = 19$ bytes

sizeof function result: 120 bytes

Comparison of Manual Calculation &

sizeof() function :-

Manual Reason:-

The manual sum (19 bytes) differs from `sizeof` result (20 bytes) because of Memory Alignment (Padding). The compiler adds 1 bytes of extra "padding" so that, the structure size is a multiple of the largest member's alignment requirement (like in above examples, `int` and `float` require 4-byte Alignment).

→ Both C & C++ give identical result in `sizeof()` function (20 bytes), because both languages handle the structure identically at memory level. Manual calculation doesn't match actual allocated size because it fails to account for hidden padding added by compiler to optimize CPU memory Access.